BUIzz

BUIzz - WebSec-Lab, GitHub.

- Published: date not stated
- Original: <https://github.com/WebSec-Lab/BUIzz>
- Preserved from: https://github.com/WebSec-Lab/BUIzz (git) on 2026-08-19
- Repository commit: 4d573c437f0ca88824b9a15af66255ad147a5f49
- Licence: see the repository

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/WebSec-Lab/BUIzz>
- Commit: `4d573c437f0ca88824b9a15af66255ad147a5f49`
- Documents preserved: 2

README.md

Blob `2fc2e2127fc1` , 8320 bytes, at commit `4d573c437f0c` .

BUIzz

BUIzz is a browser testing framework designed to find policy enforcement bugs by simulating BUI (Browser User Interface)-level user interactions at the OS level.

[image: BUIZZ Overview]

> **Minimal working example** — see `example/README.md` for a step-by-step walkthrough using the bundled Brave binary.

Repository Structure

```
BUIZZ/
    crawler/            # User interaction study data
    example/            # Minimal working example
    fuzzer/             # Core fuzzer
        fuzzer.py       #   Entry point
        user_scenario_gen.py  # Scenario generator
        lib/            #   Browser drivers & interaction helpers
        scenario/       #   Generated scenario JSON files
        test_list/      #   Corpus URL lists per policy
        browser_info/   #   Browser executable paths & UI configuration
    server/             # Per-policy report servers
        samesite/
        csp/
        coop/
        hsts/
        pp/
        rp/
        sandbox/
        xfo/
    fuzzerV2/           # Extended fuzzer variant
    scheduler/          # Distributed fuzzing coordinator
    bugs/               # Analyzer output
    Figure/             # Paper figures
    safe_error/         # Baseline-error URL list
    analyzer.py         # Inconsistency analyzer
    base_line.py        # Baseline collector
    base_lineV2.py      # Extended baseline
    deduping.py         # Bug deduplicator
    makeDB.py           # Database initializer
    schema.sql          # Database schema
    setup.ps1           # Environment setup
    certs.ps1           # TLS certificate generation
```

Tested Environments

The artifact was developed and validated on the following setup:

Component	Tested Version
OS	Windows 11 23H2 (64-bit)
Python	3.11.x
Playwright	1.44+
MySQL	8.0.x

Component	Tested Version
Docker Desktop	4.x
Chrome	138
Firefox	139
Edge	136
Opera	118
Brave	1.79
Whale	4.32

Hardware: 16 GB RAM minimum, 50 GB free disk space recommended

Prerequisites

The following programs must be installed before running BUIZZ:

Program	Version	Purpose
Windows 10/11	64-bit	Required OS — pywinauto and pyautogui drive real OS-level input
Python	3.11+	All fuzzer, analyzer, and server scripts
Docker Desktop	Latest	Runs the per-policy report servers via `docker compose`
MySQL	8.0+	Stores test results (`event_entry` table); must be running on `localhost:3306` with user `root` / password `1234`
mkcert	Latest	Generates locally-trusted TLS certificates (`certs.ps1` installs it automatically)
Browsers	See below	The actual browsers under test

Browsers required (install only those you intend to test):

Browser	Installer
Google Chrome	https://www.google.com/chrome
Microsoft Edge	Pre-installed on Windows 10/11
Mozilla Firefox	https://www.mozilla.org/firefox
Brave	Portable binary bundled in `example/brave-v1.80.120-win32-x64/` — no installation needed
Opera	https://www.opera.com
Naver Whale	https://whale.naver.com

Setup

1. Install dependencies

Run `setup.ps1` **as Administrator** in PowerShell. It installs all required Python packages and automatically updates the hosts file with the required domains.

```
# Auto-detect the local IP
PowerShell -ExecutionPolicy Bypass -File setup.ps1

# Specify an IP explicitly (e.g. when running the server on a separate machine)
PowerShell -ExecutionPolicy Bypass -File setup.ps1 10.20.23.182
```

If no IP is provided, the script detects the primary non-loopback IPv4 address of the current machine automatically.

The script installs the following packages and runs `playwright install`:

```
playwright  psutil  pywinauto  pyautogui  webdriver-manager  mysql-connector-python
```

> **Manual alternative** — if you prefer to install packages individually: ``` `bash pip install playwright psutil pywinauto pyautogui webdriver-manager mysql-connector-python python -m playwright install` ` Then add the following entries to `C:\Windows\System32\drivers\etc\hosts` : ` `<your-ip> leak.test <your-ip> adition.com <your-ip> attacker.test <your-ip> attacker.com <your-ip> victim.com` ```

2. Generate TLS certificates

Run `certs.ps1` to install mkcert and generate TLS certificates for all HTTPS servers:

```
PowerShell -ExecutionPolicy Bypass -File certs.ps1
```

3. Create the database

```
python makeDB.py
```

Collector

The `crawler` directory contains an XLSX file that comprehensively collects the user interactions gathered in our study.

Simulator

Server

Navigate to the target policy's server directory and start it using Docker Compose:

```
cd server/samesite
docker compose up
```

Available server directories: `csp`, `samesite`, `pp`, `coop`, `hsts`, `rp`, `xfo`, `sandbox`

Scenario Generation

User interaction scenarios are generated using `user_scenario_gen.py` in the `fuzzer` directory.

```
python fuzzer/user_scenario_gen.py -s samesite -b chrome -d 1
```

Option	Description
`-b`	Browser (`chrome`, `firefox`, `edge`, `opera`, `brave`, `whale`)
`-s`	Security policy (`samesite`, `csp`, `sandbox`, `pp`, `coop`, `hsts`, `rp`, `xfo`)
`-d`	Depth (`1` = single interaction, `2` = two-interaction combination)

OS-Level Simulation

The fuzzer is executed via `fuzzer.py`:

```
python fuzzer/fuzzer.py -s samesite -b chrome
```

Detector

Baseline collection

To record the pre-interaction state, run `base_line.py` to capture baseline enforcement behavior before any user interaction is performed:

```
python base_line.py -s samesite -b chrome
```

Analysis

Run `analyzer.py` to compare pre-interaction and post-interaction enforcement outcomes and identify inconsistencies:

```
python analyzer.py -s samesite -b chrome
```

Add `--lenient` for higher recall (may increase noise):

```
python analyzer.py -s samesite -b chrome --lenient
```

Results are written to `bugs/<policy>/interaction_diff_<browser>.txt`.

Deduplication

Run `deduping.py` to deduplicate flagged inconsistencies and group them into distinct root-cause bugs:

```
python deduping.py -s samesite
python deduping.py -s samesite -b chrome       # single browser
python deduping.py -s samesite --merge-tags    # merge by root cause
python deduping.py -s samesite -d 1            # depth-1 scenarios only
python deduping.py -s samesite -d 2            # depth-2 scenarios only
```

Database Schema

The `event_entry` table stores all fuzzing and baseline records. See `schema.sql` for the full annotated schema, or run `python makeDB.py` to create the database automatically.

Column	Description
`browser_name`	Browser under test (`chrome` , `firefox` , `brave` , ...)
`scenario_id`	Scenario file name (e.g. `0_DEPTH1.json`); NULL for baseline records
`corpus`	Full URL of the corpus page loaded
`event_type`	`corpus` (baseline) or `interaction` (post-interaction result)
`corpus_type`	Security policy (`samesite` , `csp` , `referrer-policy` , ...)
`leak`	Leak channel that triggered the report (e.g. `a-href` , `fetch` , `img`)
`violation`	Enforcement outcome reported by the server (e.g. `lax` , `1` , empty)
`interaction`	Human-readable label of the simulated interaction; NULL for baseline
`timestamp`	UTC timestamp of record insertion

Citation

You can cite our paper with the following bibtex entry.

```
@INPROCEEDINGS{jung:usenixsec:2026,
  author = {Jung, Mingi and Kim, Donggyu and Kim, Mijung and Wi, Seongil},
  title = {{BUIzz}: Finding Policy Enforcement Bugs via Interaction Simulation on the Browser User Interface},
  booktitle = {In Proceedings of the {USENIX} Security Symposium},
  pages = {4961--4980},
  year = 2026
}
```

example/README.md

Blob `4151e40e1377`, 7330 bytes, at commit `4d573c437f0c`.

Minimal Working Example — Brave

This directory provides a self-contained environment to run a complete BUIZZ pipeline using **Brave** without installing any additional browser.

> [!IMPORTANT] **Every command in this document must be run from the `example/` directory.** Open a terminal, `cd` into `example/`, and keep it there for all steps. ``` `powershell cd path\to\BUIZZ\example` ```

Directory Structure

```
example/
    setup.py              # Downloads Brave + seeds the pre-configured profile
    brave-profile-template/ # Pre-configured Brave profile (Shields off, English UI)
    mini_scenario.py      # Deploys pre-selected bug scenarios into fuzzer/scenario/
    samesite_split/       # Pre-selected scenarios for SameSite split-view bug (bug_20)
    csp_split_blob/       # Pre-selected scenarios for CSP split-view blob: bug (bug_07)
    csp_split_data/       # Pre-selected scenarios for CSP split-view data: bug (bug_06)
```

Why Brave?

Among the six browsers tested, Brave is the only one suitable for a pinned reproducible example. Chrome, Firefox, and Opera auto-update on launch; Edge and Whale do not offer older version archives on their official sites. Brave alone provides versioned standalone ZIP archives on GitHub Releases with no auto-update.

Prerequisites

```
# 1. Install Python packages and configure hosts file (run as Administrator)
PowerShell -ExecutionPolicy Bypass -File ..\setup.ps1

# 2. Generate TLS certificates
PowerShell -ExecutionPolicy Bypass -File ..\certs.ps1

# 3. Create the MySQL database
python ..\makeDB.py

# 4. Download and configure the portable Brave binary
python setup.py
```

`setup.py` does three things:

1. Downloads `brave-v1.80.120-win32-x64.zip` (~200 MB) from the official Brave

GitHub release and extracts it.

1. Seeds a **pre-configured, isolated Brave profile** at `example/brave-profile/`

by copying `brave-profile-template/` — Shields already disabled and the UI forced to English. (Skipped if `brave-profile/` already exists, so re-running never wipes your state.)

1. Writes the launch command into `..\fuzzer\browser_info\browser_info.json`,

pinning the extracted `brave.exe`, the dedicated `--user-data-dir`, and `--lang=en-US`.

Because Brave runs from its own `--user-data-dir`, it never collides with any system-installed Brave, and **no manual browser configuration is required.**

Docker Desktop must be running before Step 1 below.

Brave configuration (pre-applied — no action needed)

Brave's built-in Shields would otherwise interfere with the fuzzer's cross-site requests, so they must be turned off. **You don't need to do this manually** — the bundled profile in `brave-profile-template/` already has the required settings, and `setup.py` copies it into `example/brave-profile/`. The fuzzer then launches Brave against that profile.

The pre-applied settings are:

Setting	Location	Value
Trackers & ads blocking	`brave://settings/shields`	**Disabled**
Upgrade connections to HTTPS	`brave://settings/shields`	**Disabled** (prevents forced HTTPS redirect on local test domains)
Block fingerprinting	`brave://settings/shields`	**Disabled**
Block cookies	`brave://settings/shields`	**Allow all cookies**

Setting	Location	Value
Startup behavior	`brave://settings/onStartup`	**Open the New Tab page**
Display language	`brave://settings/languages`	**English (United States)**

<details> <summary>Optional — set these manually instead</summary>

If you prefer to configure Brave yourself, launch the bundled binary with its dedicated profile:

```
.\brave-v1.80.120-win32-x64\brave.exe --user-data-dir=".\brave-profile" --lang=en-US --no-first-run
```

Then apply the values in the table above under `brave://settings/shields` and `brave://settings/onStartup`:

[image: Brave Shields settings] [image: Brave On Startup setting]

</details>

Step 1 — Deploy pre-selected scenarios

Choose which bug to test and run the corresponding command:

Argument	Policy	Bug
`1`	SameSite	Split-view bug (bug_20)
`2`	CSP	Split-view `blob:` bug (bug_07)
`3`	CSP	Split-view `data:` bug (bug_06)

```
python mini_scenario.py 1   # SameSite split-view bug  (bug_20)
python mini_scenario.py 2   # CSP split-view blob: bug (bug_07)
python mini_scenario.py 3   # CSP split-view data: bug (bug_06)
```

The remaining steps use the policy that matches your choice: `samesite` for argument `1`, `csp1` for arguments `2` or `3`.

Step 2 — Start the report server

```
# For SameSite (scenario 1)
cd ..\server\samesite\corpus && docker compose up -d && cd ..\..\..\example

# For CSP (scenario 2 or 3)
cd ..\server\csp && docker compose up -d && cd ..\..\example
```

Step 3 — Collect the baseline

```
# SameSite
python scenario_base_line.py -b brave -s samesite

# CSP
python scenario_base_line.py -b brave -s csp1
```

`scenario_base_line.py` reads the scenario files already deployed in `fuzzer/scenario/brave/<policy>/` and visits only those corpus URLs using Chrome as the baseline browser (automatically selected).

Step 4 — Run the fuzzer

```
# SameSite
python ..\fuzzer\fuzzer.py -s samesite -b brave

# CSP
python ..\fuzzer\fuzzer.py -s csp1 -b brave
```

> Do **not** move the mouse or switch windows while the fuzzer is running.

Step 5 — Analyze inconsistencies

```
# SameSite
python ..\analyzer.py -s samesite -b brave

# CSP
python ..\analyzer.py -s csp -b brave
```

Expected output:

```
[strict] flagged N/M interaction rows
[+] Results written to bugs/<policy>/interaction_diff_brave.txt
```

Step 6 — Deduplicate results

```
python ..\deduping.py -s samesite -b brave   # or -s csp1
```

Expected output:

```
============================================================
 BUIZZ deduplication - policy=samesite  browser=brave
============================================================
 Raw inconsistency records : 12
 Distinct bugs (after dedup): 4

 Bug #01: (Open link in a split window, <a>, https:)
        browsers : brave
        scenario : 8_DEPTH1.json
        records  : 3

 Bug #02: (Open link in background tab, <a>, https:)
        browsers : brave
        scenario : 1_DEPTH1.json
        records  : 5

 Bug #03: (Open link in new tab, <a>, https:)
        browsers : brave
        scenario : 11_DEPTH1.json
        records  : 2

 Bug #04: (Open link in new window, <a>, https:)
        browsers : brave
        scenario : 13_DEPTH1.json
        records  : 2

 Total distinct bugs: 4
```

The following is a sample record from the analyzer's inconsistency output — an interaction where Brave's behaviour differed from the baseline:


```
{'browser_name': 'brave', 'scenario_id': '8_DEPTH1.json', 'interaction': 'Open link in a split window', ... ,'leak': 'a-href', 'vic
```


The `violation: strict` field means the `SameSite=Strict` cookie was transmitted — a behaviour that should be blocked, confirming a SameSite policy bypass.

This bug has been assigned **CVE-2025-48980**.